



Anil Seth

PackageKit—Overcoming Prejudice

My first reaction to the PackageKit UI, after its initial release on Fedora, was negative. It was prejudice at first sight.

I did not see any advantage in reinventing the wheel. The only GUI package manager I used was Synaptic on Ubuntu. Usually, the GUI package managers seemed to be too limiting and, besides, “yum -y update” is hardly a problem to remember. Even searching for packages via Yum was not a problem. Using *pacman* or *apt* is equally easy.

I did not see the need to unify the backends of package managers. For all practical purposes, only *rpm* or *dpkg* formats mattered and there seemed to be little reason for someone to create one more. Users of distributions like Gentoo and Arch Linux were not particularly interested in a GUI frontend for their package management applications.

Even the controversy that arose with a proposed change in the package installation policy of Fedora 12 (https://bugzilla.redhat.com/show_bug.cgi?id=534047) did not raise a doubt in my mind that there was more to PackageKit than what I had assumed. I continued to ignore PackageKit.

While I would have loved greater simplicity in package management, the solutions from various distributions seemed to be simplistic GUIs. Had Synaptic been integrated into Fedora, I would have been far happier.

It was while trying to understand how the PolicyKit framework was being integrated into the distributions that I realised that I had completely misunderstood PackageKit.

My new understanding was that it was not a reinvention of the wheel but, rather, an approach to the same issue from a different perspective. The philosophy of PolicyKit is that the user interface should not run in a privileged mode. This goes against the common assumption that package management should require administrative privileges. Hence, an application like Synaptic runs as a super-user. If the PolicyKit framework is to be used, package management applications must be split into a server and a client. The server will be a privileged application, which will use PolicyKit framework and D-Bus to control access rights. The client application just does not worry about access controls at all. The access controls will be managed by policy files and the PolicyKit framework.

We can go a step further and recognise that the core operations required of a package management server are the same, or similar. Implementation details differ. Hence, instead of creating a PackageKit server only for Yum, it would be better to define a common interface for the client. This implies

that rather than recreate the functionality of Yum or extend Yum to support PolicyKit, a separate PackageKit server is written, which then has a thin layer for talking to Yum, apt or whatever else the case may be.

Unfortunately, what we as users saw was the GUI, which was disappointing. However, nothing prevents Synaptic-like applications to use the PackageKit server and appeal to a different audience. The implications are obvious. Which set of operations are to be done by whom, can be determined by the owner of the system. Package managers can offer a default policy, which may be modified by the distribution developers and which may finally be modified by the administrator of the system.

The default policy is available in `/usr/share/polkit-1/actions/org.freedesktop.packagekit.policy`. If the default is over-ridden by the distribution, it will be available in `/var/lib/polkit-1/localauthority/10-vendor.d`. The default policy requires administrative privileges on the active console to be able to install any applications. I could not install an application or update the system using the GUI application from a diskless workstation!

However, updating a system can be done by any active console user, a fact I had not noticed as I was relying only on Yum to keep my system current. You can find the various actions and the default access controls in the [org.freedesktop.packagekit.policy](https://www.freedesktop.org/software/polkit/policy) file mentioned above.

An interesting option would be to integrate the PackageKit server with *pacman* for Arch Linux instead of minimal functionality with ALPM, as it is at present.

What about Ubuntu? Here's an indication from the main page of Software Center, the new package management application for Ubuntu. “Ubuntu Software Center is implemented using Python, GTK, and Aptdaemon (launchpad.net/aptdaemon), and may use PackageKit for some components.”

I am not inclined to give up using Yum (or *pacman*) and use the GUI instead. However, my appreciation of what is being done with PackageKit has gone up. If Synaptic is modified to work with the PackageKit server, I may even switch to the GUI. 

About Anil Seth

The author works as a consultant. Prior to consulting, Anil was a professor at Padre Conceicao College of Engineering (PCCE) in Goa, managed IT and imaging solutions for Phil Corporation Limited (Goa), supported domestic customers for Tata Burroughs/TIL, and researcher with IIT-K and Indian Institute of Geomagnetism (Mumbai). You can find him online at <http://sethanil.com> and reach him via email at anil@sethanil.com.

Usually, GUI package managers seem to be too limiting...